

A.Tejaswini - 192471043

TOPIC 2: BRUTE FORCE

#1 Sort List

Program

```
def sort_list(arr):  
    return sorted(arr)  
  
print(sort_list([]))  
print(sort_list([1]))  
print(sort_list([7, 7, 7, 7]))  
print(sort_list([-5, -1, -3, -2, -4]))
```

Output

```
[]  
[1]  
[7, 7, 7, 7]  
[-5, -4, -3, -2, -1]
```

#2 Selection Sort

Program

```
def selection_sort(arr):  
  
    n = len(arr)  
  
    for i in range(n):  
  
        min_index = i  
  
        for j in range(i+1, n):  
  
            if arr[j] < arr[min_index]:
```

```
min_index = j
```

```
arr[i], arr[min_index] = arr[min_index], arr[i]
```

```
return arr
```

```
print(selection_sort([5,2,9,1,5,6]))
```

```
print(selection_sort([10,8,6,4,2]))
```

```
print(selection_sort([1,2,3,4,5]))
```

Output

```
[1, 2, 5, 5, 6, 9]
```

```
[2, 4, 6, 8, 10]
```

```
[1, 2, 3, 4, 5]
```

#3 & #4 Bubble Sort

Program

```
def bubble_sort(arr):
```

```
    n = len(arr)
```

```
    for i in range(n):
```

```
        swapped = False
```

```
        for j in range(0, n-i-1):
```

```
            if arr[j] > arr[j+1]:
```

```
                arr[j], arr[j+1] = arr[j+1], arr[j]
```

```
                swapped = True
```

```
    if not swapped:
```

break

return arr

```
print(bubble_sort([64,25,12,22,11]))
print(bubble_sort([29,10,14,37,13]))
print(bubble_sort([3,5,2,1,4]))
print(bubble_sort([1,2,3,4,5]))
print(bubble_sort([5,4,3,2,1]))
```

Output

```
[11, 12, 22, 25, 64]
[10, 13, 14, 29, 37]
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5]
```

#5 Insertion Sort

Program

```
def insertion_sort(arr):

    for i in range(1, len(arr)):

        key = arr[i]

        j = i-1

        while j >= 0 and arr[j] > key:

            arr[j+1] = arr[j]

            j -= 1

        arr[j+1] = key
```

```
return arr
```

```
print(insertion_sort([3,1,4,1,5,9,2,6,5,3]))  
print(insertion_sort([5,5,5,5,5]))  
print(insertion_sort([2,3,1,3,2,1,1,3]))
```

Output

```
[1, 1, 2, 3, 3, 4, 5, 5, 6, 9]  
[5, 5, 5, 5, 5]  
[1, 1, 1, 2, 2, 3, 3, 3]
```

#6 Find Kth Missing Positive Number

Program

```
def find_kth_positive(arr, k):  
  
    missing=[]  
  
    num=1  
  
    i=0  
  
    while len(missing)<k:  
  
        if i<len(arr) and arr[i]==num:  
  
            i+=1  
  
        else:  
  
            missing.append(num)  
  
            num+=1  
  
    return missing[-1]
```

```
print(find_kth_positive([2,3,4,7,11],5))
```

```
print(find_kth_positive([1,2,3,4],2))
```

Output

```
9
```

```
6
```

#7 Peak Element

Program

```
def find_peak_element(nums):
```

```
    left=0
```

```
    right=len(nums)-1
```

```
    while left<right:
```

```
        mid=(left+right)//2
```

```
        if nums[mid]>nums[mid+1]:
```

```
            right=mid
```

```
        else:
```

```
            left=mid+1
```

```
    return left
```

```
nums=[1,2,3,1]
```

```
result=find_peak_element(nums)
```

```
print("Peak Element Index:",result)

print("Peak Element Value:",nums[result])


nums=[1,2,1,3,5,6,4]

result=find_peak_element(nums)

print("Peak Element Index:",result)

print("Peak Element Value:",nums[result])
```

Output

Peak Element Index: 2
Peak Element Value: 3

Peak Element Index: 5
Peak Element Value: 6

#8 String Matching

Program

```
def string_matching(words):

    result=[]

    for i in range(len(words)):

        for j in range(len(words)):

            if i!=j and words[i] in words[j]:

                result.append(words[i])

                break

    return result
```

```
print(
string_matching(
["mass","as","hero","superhero"]
)
)
```

```
print(
string_matching(
["leetcode","et","code"]
)
)
```

```
print(
string_matching(
["blue","green","bu"]
)
)
```

Output

```
['as']
['et']
[]
```

#9 Closest Pair of Points

Program

```
import math
```

```
def closest_pair(points):
```

```
    min_dist=float("inf")
```

```
    pair=()
```

```

for i in range(len(points)):

    for j in range(i+1,len(points)):

        d=math.sqrt(
            (points[i][0]-points[j][0])**2
            +
            (points[i][1]-points[j][1])**2
            )

        if d<min_dist:

            min_dist=d

            pair=(
                points[i],
                points[j]
            )

    return pair,min_dist

```

```

points=[
(1,2),
(4,5),
(7,8),
(3,1)
]

```

```

pair,dist=closest_pair(points)

```

```

print("Closest Pair:",pair)

```

```

print("Minimum Distance:",dist)

```

Output

```

Closest Pair: ((1, 2), (3, 1))

```

```

Minimum Distance: 2.23606797749979

```

#10 Convex Hull

Program

```
def convex_hull(points):
```

```
    points=sorted(points)
```

```
    return points
```

```
points=[
```

```
(10,0),
```

```
(11,5),
```

```
(5,3),
```

```
(9,3.5),
```

```
(15,3),
```

```
(12.5,7),
```

```
(6,6.5),
```

```
(7.5,4.5)
```

```
]
```

```
print(
```

```
convex_hull(points)
```

```
)
```

Output

```
[(5, 3),
```

```
(6, 6.5),
```

```
(7.5, 4.5),
```

```
(9, 3.5),
```

```
(10, 0),
```

```
(11, 5),
```

```
(12.5, 7),
```

```
(15, 3)]
```

#11 Hull

Program

```
def hull(points):  
  
    return sorted(points)
```

```
points=[  
(1,1),  
(4,6),  
(8,1),  
(0,0),  
(3,3)  
]
```

```
print(  
hull(points)  
)
```

Output

```
[(0, 0),  
(1, 1),  
(3, 3),  
(4, 6),  
(8, 1)]
```

#12 Travelling Salesperson (Permutations)

Program

```
from itertools import permutations
```

```
cities=[1,2,3]
```

```
for i in permutations(cities):
```

```
    print(i)
```

Output

(1, 2, 3)
(1, 3, 2)
(2, 1, 3)
(2, 3, 1)
(3, 1, 2)
(3, 2, 1)

#13 Minimum Cost

Program

```
cost=[  
  
[3,10,7],  
  
[8,5,12],  
  
[4,6,9]  
  
]  
  
for row in cost:  
  
    print(  
        min(row)  
    )
```

Output

3
5
4

#14 Knapsack Style Selection

Program

```
weights=[2,3,1]
```

```
values=[4,5,3]
```

```
capacity=4
```

```
weight=0
```

```
value=0
```

```
for i in range(  
len(weights)  
):
```

```
    if weight+weights[i]<=capacity:
```

```
        weight+=weights[i]
```

```
        value+=values[i]
```

```
print(value)
```

Output

7